

ENGIAI: A MULTI-AGENT FRAMEWORK AND BENCHMARK SUITE FOR LLM-DRIVEN ENGINEERING DESIGN

PREPRINT

✉ **Gioele Molinari***

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland
gioelemo@ethz.ch

✉ **Florian Felten**

IDEAL, ETH Zurich, Zurich, Switzerland
Autom8.build, Brussels, Belgium

✉ **Soheyl Massoudi**

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland

✉ **Mark Fuge**

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland

ABSTRACT

Large Language Model (LLM) agents are increasingly applied to engineering design tasks, yet existing evaluation frameworks do not adequately address multi-agent systems that combine simulation, retrieval, and manufacturing preparation. We introduce a benchmark suite with three evaluation dimensions: (1) a workflow benchmark with seven prompt styles targeting distinct cognitive demands—including direct tool use, semantic disambiguation, conditional branching, and working-memory tasks; (2) a Retrieval-Augmented Generation (RAG) benchmark with gated scoring isolating retrieval contributions to parameter selection; and (3) an High Performance Computing (HPC) benchmark evaluating end-to-end ML training orchestration on a SLURM cluster. Alongside the benchmark we present ENGIAI, a Multi-Agent System (MAS) reference implementation built on LangGraph that operationalizes the benchmark by coordinating seven specialized agents through a supervisor architecture, unifying topology optimization, document retrieval, HPC job orchestration, and 3D printer control. Across four LLM backends and two EngiBench problems, proprietary models achieve 96–97% average task completion on Beams2D, while open-source 4B-parameter models reach 55–78%, with clear generational improvement. Conditional branching proves most challenging, with task completion dropping to 20–53% for the conditional style on Photonics2D. RAG gating confirms near-perfect retrieval-augmented scores (≈ 1.0) versus near-zero without retrieval, validating the evaluation design. On HPC orchestration, one model completes all pipeline steps in 100% of runs while another drops to 50%, revealing that multi-step instruction following degrades over long-running workflows.

Keywords multi-agent systems · large language models · topology optimization · retrieval-augmented generation

1 Introduction

Engineering design is inherently complex, spanning conceptual layout, simulation-driven optimization, and manufacturing preparation, each relying on specialized software and iterative refinement [1]. In recent years, various Machine Learning (ML) models have been adopted to streamline the generation of design candidates, particularly through inverse design and surrogate models [2]. While these models can generate high-quality designs, they often function as “black boxes” that are inflexible and difficult for non-ML experts to interact with or interpret.

*Corresponding author.

Concurrently, the proliferation of Large Language Models (LLMs), specifically in the form of chatbots such as ChatGPT [3], has enabled users to interact naturally with complex systems through conversational interfaces. This creates an opportunity: LLM agents can serve as a natural-language bridge between engineers and specialized design tools, making ML-driven workflows accessible without requiring deep expertise in the underlying models.

This paper introduces a benchmark suite for evaluating LLM-driven engineering workflows along three dimensions—workflow execution, retrieval-augmented parameter selection, and High Performance Computing (HPC) training orchestration—together with ENGIAI, a multi-agent reference implementation that operationalizes the benchmark. Accessible through a web-based interface, ENGIAI coordinates multiple engineering tools through natural-language interaction, with a modular architecture that allows new capabilities to be added via additional agents or tool APIs. The benchmark suite and reference implementation together evaluate the following research questions:

RQ1 - Workflow Performance: Which cognitive demands—explicit parameters, qualitative descriptions, semantic disambiguation, conditional branching, derived computation, and multi-export tracking—most degrade workflow task completion and design quality, across proprietary and open-source LLM backends?

RQ2 - Model Robustness: How robust is design performance to LLM model choice and configuration?

RQ3 - Tool Usage: How does tool-calling efficiency relate to design performance—are correct designs and efficient tool use co-occurring competences?

RQ4 - RAG Improvement: How much does retrieval contribute to parameter selection accuracy, and can the contribution be cleanly isolated from parametric memorization?

RQ5 - HPC Orchestration: How reliably do current proprietary LLM agents complete long-horizon HPC training pipelines, and at which pipeline stages does multi-step instruction-following degrade?

We contribute (1) a benchmark suite comprising seven workflow prompt styles targeting distinct cognitive demands—direct tool use, semantic disambiguation, conditional branching, derived parameter computation, and working-memory tasks—evaluated across two engineering problems and four LLM backends; (2) a gated RAG scoring mechanism that isolates the contribution of document retrieval to parameter selection, preventing agents from earning credit through prior knowledge or alternative information channels; (3) an HPC training benchmark evaluating end-to-end orchestration of generative model training on SLURM clusters—a capability not addressed by existing agent benchmarks; and (4) a LangGraph [4]-based multi-agent reference implementation that operationalizes the benchmark, integrating topology optimization, RAG, HPC orchestration, and 3D printer control through a supervisor architecture.

The source code and evaluation framework are planned for public release. A video demonstration of the system is also available.

2 Related Work

Engineering design is classically decomposed into the phases of task clarification, conceptual design, embodiment design, and detail design culminating in manufacturing preparation [5], with iteration across and within these phases consistently identified as a defining characteristic of design [1]. Modern ML-augmented design adds two enabling layers: large-scale *training infrastructure* for surrogate and generative models (e.g., EngiOpt [6]), and *agentic coordination* between heterogeneous tools, recently formalized as Intelligent Design 4.0 [7]. We organize ENGIAI around a representative selection of six capabilities that map onto these phases: multi-agent orchestration handles natural-language entry and inter-tool routing (task clarification); physics simulation and topology optimization cover the analysis side of embodiment and detail design; RAG prevents parameter hallucination by grounding decisions in domain documents; STL export supports the manufacturing handoff; HPC orchestration enables ML-based design training; and structured benchmarking isolates model capability from prompt sensitivity. Existing systems cover subsets of these phases but, to our knowledge, no prior work addresses all of them within a single framework (Table 1).

2.1 Multi-agent orchestration

Recent surveys [8, 9] document the rapid growth of agentic AI—systems employing LLM-based agents to accomplish complex task sequences—across scientific domains. General-purpose frameworks such as AutoGen [10], CrewAI [11], the OpenAI Agents SDK [12], and LangGraph [4] provide multi-agent orchestration infrastructure, while the AI Co-Scientist [13] demonstrated that structured agent orchestration can outperform single-model reasoning on complex research tasks. Jiang *et al.* [7] situate these developments within a four-stage evolution of intelligent design, proposing an ontological framework for multi-agent end-to-end design automation (ID 4.0). In mechanical engineering, several MASSs have applied these ideas to specific design tasks: topology optimization and Finite Element Analysis (FEA) [14, 15, 16],

industrial structural analysis in aerospace [17], automotive conceptual design [18], early-stage product design [19], mechatronics [20], and design-state graph coordination across multiple LLM backends [21]. However, these systems focus on conceptual design phases and do not extend to manufacturing export, HPC-based training, or retrieval-augmented parameter selection.

2.2 Tool integration and manufacturing

Bridging the gap from design to physical production requires standardized tool interfaces and manufacturing-aware agents. The MCP [22] addresses the former by standardizing tool integration across heterogeneous services, though Wang *et al.* [23] show that even frontier models struggle with complex multi-step tool orchestration. On the manufacturing side, Jadhav and Farimani [24] demonstrate a hierarchical agent system for real-time Fused Deposition Modeling (FDM) print monitoring, but their pipeline does not include simulation or design optimization. For standardized access to engineering problems, Felten *et al.* [6] introduce EngiBench, providing a unified Python Application Programming Interface (API) for common mechanical engineering problems and ML-based design optimization via the companion EngiOpt framework. No existing system integrates tool orchestration with both physics-based simulation and retrieval-augmented reasoning.

2.3 Retrieval-augmented engineering

RAG [25, 26] enables agents to ground their outputs in domain-specific documents rather than relying solely on parametric knowledge. Without retrieval, agents risk hallucinating parameter values or falling back on training-data defaults—a failure mode that is particularly consequential in engineering, where incorrect parameters produce physically invalid designs. Chandrasekhar *et al.* [27] pair Llama2-7B with indexed Additive Manufacturing (AM) papers to achieve competitive performance with GPT-4 on metal additive manufacturing queries. Khanghah *et al.* [28] extend multimodal RAG to zero-shot anomaly detection in laser powder bed fusion, showing that retrieval from scientific literature improves classification accuracy by 12% over non-retrieval baselines. These applications demonstrate the value of retrieval for individual engineering tasks, but none embed RAG within a multi-agent simulation pipeline where retrieved parameters directly drive downstream tool calls.

2.4 Agent benchmarking

Evaluating agent systems requires benchmarks that capture the multi-step, tool-intensive nature of real workflows. Recent surveys [29] categorize agent evaluation along dimensions such as tool-calling accuracy, multi-step planning, and task completion. Tool-calling benchmarks—ACEBench [30], ToolSandbox [31], BFCL [32], and ToolBench [33]—consistently reveal performance gaps between open-source and proprietary models. Broader agent evaluations such as AgentBench [34], AgentBoard [35], and ScienceAgentBench [36] assess reasoning across diverse environments but target single-agent tasks. Within engineering, FDM-Bench [37] evaluates LLMs on FDM tasks, and Zhou *et al.* [38] introduce a hierarchical benchmark revealing that models struggle with open-ended modeling tasks. For multi-turn professional tasks, τ -bench [39] reports <50% task success even for state-of-the-art models. However, none of these benchmarks address multi-agent engineering workflows that combine simulation, retrieval, and HPC orchestration. Such workflows expose failure modes absent from standard tool-calling evaluations: agents must chain numeric outputs from one tool into downstream decisions (e.g., reading a simulation result to select conditional parameters), maintain numerical precision when prompts contain competing plausible values for the same parameter, and track multiple parameter sets across sequential tool calls—working-memory and semantic-grounding demands that single-call benchmarks do not elicit.

3 Methods

3.1 System Architecture

The ENGIAI system follows a hierarchical supervisor pattern (Figure 1): a central supervisor agent receives each user prompt, classifies the intent, and routes execution to the appropriate specialist agent. Each specialist operates as a self-contained LangGraph state machine—receiving a task, invoking its domain-specific tools, and returning results to the supervisor. The supervisor pattern is motivated by tool-set scalability: prior work shows that LLMs degrade in tool-selection accuracy as the number of available tools grows [23], so decomposing the tool space across specialized agents keeps each agent’s effective tool list small and bounded. The supervisor can chain multiple agents sequentially (e.g., RAG agent to retrieve parameters, then engineering agent to run the optimization) and enforces termination when repeated re-routing is detected. The framework provides stateful agent graphs with conditional routing, built-in tool binding, human-in-the-loop interrupts, and checkpoint-based persistence via LangChain [42].

Table 1: Representative selection of LLM-relevant engineering capabilities covered by ENGIAI versus prior work, grouped into *Capabilities*, *Infrastructure*, and *Evaluation*. ✓ = supported, × = not supported. Systems are split into engineering agent systems (top) and evaluation benchmarks (bottom).

	Capabilities				Infra.	Eval.
	Multi-Agent	RAG	Simulation	Mfg. Export	HPC	Benchmark
<i>Engineering Agent Systems</i>						
DSG-MAS [21]	✓	×	✓	×	×	✓
MechAgents [14]	✓	×	✓	×	×	×
FEAGPT [15]	×	×	✓	×	×	×
LLM3DPrint [24]	✓	×	×	✓	×	×
DUCTILE [17]	×	×	✓	×	×	×
AMGPT [27]	×	✓	×	×	×	×
Wang et al. [20]	✓	×	✓	×	×	×
<i>Evaluation Benchmarks</i>						
EngiBench/EngiOpt [6]	×	×	✓	×	×	✓
EngiBench [38]	×	×	×	×	×	✓
FDM-Bench [37]	×	×	×	×	×	✓
General benchmarks [30, 31, 32, 33, 23, 34, 35, 36, 39, 40, 41]	×	×	×	×	×	✓
ENGIAI (ours)	✓	✓	✓	✓	✓	✓

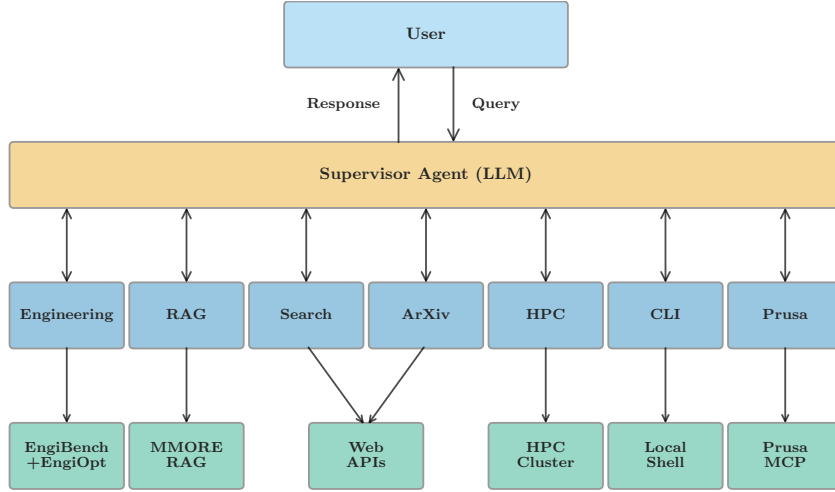


Figure 1: Multi-agent architecture. From top to bottom: the user interface, the orchestration layer (supervisor agent routing prompts), the specialization layer (seven domain agents), and the execution layer (external services and tools).

A typical design session illustrates the pattern: when a user requests a topology optimization, the supervisor routes to the *engineering agent*, which calls *create_problem* → *optimize_design* → *simulate_design* → *render_design* (or *convert_design_to_stl* for manufacturing export). If the user first asks about a parameter from a paper, the supervisor routes to the *RAG agent*, which retrieves the relevant information, then re-routes to the *engineering agent* with the extracted values.

3.2 Agent Capabilities

Table 2 summarizes the seven specialist agents and their tools. We organize them by their role in the design-to-manufacturing lifecycle.

Engineering agent. The core design agent drives topology optimization, simulation, ML-based inverse design, and Standard Tessellation Language (STL) export through the EngiBench/EngiOpt [6] stack. EngiBench provides a unified

Python API for accessing engineering design problems across multiple physics domains; EngiOpt extends it with ML-based generative design algorithms (e.g., conditional Generative Adversarial Network (cGAN)s, diffusion models) that can be trained and evaluated against dataset baselines.

Information retrieval agents. Three agents provide complementary retrieval capabilities. The *RAG agent* and *ArXiv agent* share the MMORE (Multimodal Open Retrieval Engine) [43] framework, which indexes and retrieves information from heterogeneous document types (PDFs, images, tables)—the former for document Q&A over user-indexed collections, the latter for fetching and indexing papers from arXiv. The *search agent* provides web search via the Tavily API.

Infrastructure agents. The *HPC agent* manages SLURM jobs on remote clusters via SSH. The *CLI agent* executes local shell commands (with a whitelisted set of GUI applications and human-in-the-loop confirmation step before running commands), and the *Prusa agent* controls 3D printers through a dedicated Model Context Protocol (MCP) [22] server.

Table 2: ENGIAI multi-agent architecture and tool integration.

Agent	Primary Function	# Tools
Supervisor	Request routing	–
Engineering	Optimization, ML, STL export	14
Search	Web retrieval	1
RAG	Document Q&A	5
ArXiv	Paper retrieval	5
HPC	Remote job submission	8
CLI	Command execution	2
Prusa (MCP)	3D printer control	8

3.3 Experimental Setup

3.3.1 Evaluation Problems

We evaluate on two EngiBench [6] problems spanning different physics domains.

Beams2D. A 50×100 cantilever beam topology optimization problem minimizing compliance subject to a volume fraction constraint [44, 45]. This well-studied structural optimization benchmark provides ground-truth solutions for evaluating agent correctness. Key parameters are volume fraction (*volfrac*), force distance (*forcedist*), and filter radius (*rmin*).

Photonics2D. A 120×120 photonic device inverse design problem maximizing electromagnetic field overlap for specified wavelengths, testing whether agent capabilities transfer across physics domains.

3.3.2 Models and Configuration

We evaluate four LLM backends spanning two categories: *proprietary cloud models* (GPT-5-mini (gpt-5-mini-2025-08-07) [46], Gemini-3-Flash (gemini-3-flash-preview)) and *open-source 4B-parameter models* served locally via Ollama, a lightweight inference server for open-weight models (Qwen3-4B (qwen3:4b-instruct-2507-q8_0) [47], Qwen3.5-4B (qwen3.5:4b-q8_0)). This selection enables comparison across both the proprietary–open-source divide and across model generations at fixed parameter count; all four models support structured tool calling and were chosen at cost-efficient tiers to reflect realistic deployment budgets. All calls use temperature = 0 with a fixed API seed to maximize reproducibility (see Section 5.3 for residual variation). Each model–style cell comprises 15 runs (3 random seeds $\times$ 5 dataset samples). Beams2D is evaluated on all seven prompt styles; Photonics2D on three (see Section 5.3).

3.4 Benchmark Design

We evaluate along three dimensions that test complementary system capabilities: (1) *workflow execution*—can agents reliably chain engineering tools under prompts probing distinct cognitive demands? (RQ1-RQ3) (2) *retrieval-augmented parameter selection*—does document retrieval improve engineering decisions? (RQ4) and (3) *HPC orchestration*—can agents manage long-running training pipelines on remote infrastructure? (RQ5)

3.4.1 Workflow Evaluation

We define seven prompt styles spanning distinct cognitive demands (Table 3; representative prompts in Appendix C). The first two establish baselines: FULL provides all numerical parameters for a direct optimize→simulate→render pipeline, while NATURAL uses qualitative descriptions (e.g., “lightweight” instead of $\text{volfrac} = 0.2$), requiring the agent to detect ambiguity and request human clarification.

The five W-styles extend FULL with an STL export step. Each modifies only the export instructions while keeping the optimization and simulation steps identical, isolating the effect of instruction phrasing on task completion. The styles target three categories of failure mode that standard tool-calling benchmarks do not capture.

The first category is *numerical fidelity*: can the agent propagate precise values from prompt to tool call? W-RAND tests this directly with explicit randomized export parameters (density threshold, XY scale, Z extrusion, mirror flag). W-DERIVED raises the bar by specifying parameters as derivation rules (e.g., “use the volume fraction as the density threshold”), requiring arithmetic over values the agent passed to earlier tools.

The second is *semantic grounding*: can the agent select the correct value when the prompt contains competing alternatives? W-DISTRACT presents two plausible values for the same parameter—one in a preview context, one for manufacturing export—both of which the tool accepts without error. Only the parameter-validation scorer distinguishes correct from incorrect choices, making this failure mode invisible to benchmarks that check only function signatures.

The third is *stateful reasoning*: can the agent condition its actions on runtime information? W-COND requires the agent to read a compliance value from *simulate_design* and select the corresponding parameter branch via an if/then rule. W-MULTI requires two sequential export calls with entirely different parameter sets, testing whether the agent can maintain distinct configurations in working memory rather than merging or dropping one.

Table 3: Workflow evaluation prompt styles. [†]Randomized STL parameters (*mirror_y*, *scale_xy*, *scale_z*, *threshold*); validated within ± 0.05 absolute tolerance on each parameter value to account for LLM rounding. [‡]STL parameters derived from optimization inputs.

Style	Input	Capability	Sequence
FULL	All num. params	Direct tool use	opt→sim→ren
NATURAL	Text only	Ambiguity detect.	ask_human
W-RAND	+ rand. STL [†]	Instr. adherence	opt→sim→stl
W-DERIVED	+ deriv. rules [‡]	Arithmetic	opt→sim→stl
W-DISTRACT	+ competing vals [†]	Semantic disamb.	opt→sim→stl
W-COND	+ branching [†]	Conditional	opt→sim→stl
W-MULTI	+ two exports [†]	Working memory	opt→sim→stl→stl

3.4.2 RAG Evaluation

We design four handcrafted prompts (Table 4; full text in Appendix C) with increasing retrieval demand. Unlike the workflow benchmarks, which sample from EngiBench datasets, the RAG evaluation uses fixed prompts that instruct the agent to find specific optimization parameters from indexed research papers and use them in a design task. Prompt difficulty increases from a single default parameter (P0) to mixed-source retrieval across two papers (P3).

Table 4: RAG evaluation prompts with escalating retrieval difficulty. Scoring weights are given in Table 7.

ID	Source	Target Parameters		
		volfrac	forcedist	rmin
P0	EngiBench paper [6]	0.35	—	—
P1	EngiBench API ex. [6]	0.70	0.30	—
P2	SOPTX paper [48]	0.40	—	6.0
P3	EngiBench [6] + SOPTX [48]	0.70	0.30	6.0

We evaluate under three conditions to isolate the contribution of document retrieval. RAG-on provides reference documents indexed and retrievable via *search_documents*. RAG-off removes retrieval tools entirely, forcing the agent to rely on parametric knowledge. Empty RAG is the critical ablation: the retrieval tools are available but the index is empty, testing whether the mere availability of a retrieval tool changes agent behavior—for example, by inducing the agent to trust empty results and omit parameters it would otherwise guess from training data.

To ensure the only retrieval path is *search_documents*, we disable the web search and ArXiv agents for all RAG evaluation runs.

3.4.3 HPC Training Evaluation

This benchmark tests whether the agent can orchestrate a complete ML training pipeline on remote HPC infrastructure. The expected workflow comprises four sequential steps: (1) generate a SLURM training script with the correct algorithm, problem, epochs, and seed; (2) submit the script to the cluster for remote execution; (3) monitor the job until completion; and (4) evaluate the trained model against the dataset baseline using EngiOpt metrics (Maximum Mean Discrepancy (MMD), Determinantal Point Process (DPP), Ratio of Violated Constraints (RVC), and optimality gaps Intermediate Optimality Gap (IOG), Cumulative Optimality Gap (COG), Final Optimality Gap (FOG); defined in Appendix B).

We define 10 prompts per style (seeds 1–10) with 100 fixed training epochs, using two prompt formats (Appendix C): EXPLICIT (step-by-step instructions with tool names) and NATURAL (high-level description requiring the agent to infer the workflow). We report results for the cGAN algorithm; diffusion model results are provided in Appendix D.1.

3.5 Scoring Methodology

The workflow evaluation combines three aspects into a single hierarchical score: *design quality* (65%)—a weighted combination of six sub-metrics covering geometric overlap (Intersection over Union (IoU), pixel accuracy), engineering objectives (objective match, constraint match), and manufacturability (2D connectivity, 3D watertightness; individual weights in Appendix A); *tool use* (20%)—the ratio of correctly matched calls to the maximum of optimal and actual call counts; and *task completion* (15%)—a binary indicator of whether all required tools were called successfully, with STL export styles additionally requiring parameter match within tolerances (Appendix A).

Weights reflect the evaluation priorities: design quality receives the largest share (65%) because it directly measures whether the agent produced a correct engineering artifact; tool use (20%) captures operational cost, as redundant or missing tool calls indicate inefficient agent behavior; and task completion (15%) acts as a binary gating signal that is already implicitly reflected in design quality (a failed task typically yields no designs or low-quality ones). To support alternative weighting schemes, Appendix A reports all individual sub-scores alongside the composite. Two caveats apply when interpreting design quality: (i) the 3D watertightness sub-score is structurally low across all models because our threshold-based STL extraction produces non-manifold meshes (see caption in Tables 8 and 9), so this term reflects a pipeline limitation rather than agent capability; and (ii) since the manufacturability sub-scores are only meaningful for the W-styles that invoke STL export, design quality should be compared *per style* rather than as a cross-style average.

For evaluating generative models (HPC training benchmark), we additionally report global distribution metrics—MMD for dataset fidelity and DPP for design diversity—as well as engineering performance metrics such as RVC and optimality gaps (IOG, COG, FOG), following Felten *et al.* [6]; full definitions are given in Appendix B.

RAG Scoring The RAG evaluation employs a gated scoring mechanism: parameter accuracy dimensions only contribute to the score when the agent invokes the *search_documents* tool, preventing credit for values reached through alternative means. The score combines per-parameter accuracy with a retrieval indicator (Appendix A.2).

HPC Workflow Scoring The HPC training benchmark scores workflow orchestration rather than design quality. The primary metric is a weighted composite of step completion (70%), configuration correctness (15%), and evaluation metric extraction (15%). Appendix A.3 provides the full formulation.

4 Results

4.1 Workflow Evaluation

Table 5 summarizes the Beams2D task completion (TC) and combined overall (CO) scores across all seven prompt styles and four LLM backends.²

Both GPT-5-mini and Gemini-3-Flash achieve near-perfect TC on most styles; in contrast, the open-source 4B-parameter models show varying degrees of success. Qwen3-4B achieves low task completion on most styles (55% average TC), while Qwen3.5-4B, the next-generation model at the same parameter count, reaches 78% average TC, narrowing the

²A small number of Qwen3.5-4B runs (1–2 per affected style) were lost due to JSON parsing errors in the agent’s output, resulting in $n=13$ – 14 instead of 15 for three workflow styles. All other model–style combinations have the full $n=15$.

Table 5: Workflow evaluation results for Beams2D (mean $\pm$ std). TC = Task Completion rate, CO = Combined Overall score. **Bold** = best model per metric per row.

Style	GPT-5-mini		Gemini-3-Flash		Qwen3-4B		Qwen3.5-4B	
	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$
FULL	1.00 $\pm$ 0.00	0.65 $\pm$ 0.07	0.93 $\pm$ 0.25	0.69 $\pm$ 0.10	0.00 $\pm$ 0.00	0.45 $\pm$ 0.06	0.73 $\pm$ 0.44	0.59 $\pm$ 0.11
NATURAL	0.87 $\pm$ 0.34	0.82 $\pm$ 0.30	1.00 $\pm$ 0.00	0.88 $\pm$ 0.23	0.00 $\pm$ 0.00	0.29 $\pm$ 0.05	0.33 $\pm$ 0.47	0.48 $\pm$ 0.32
W-RAND	1.00 $\pm$ 0.00	0.66 $\pm$ 0.07	1.00 $\pm$ 0.00	0.65 $\pm$ 0.09	1.00 $\pm$ 0.00	0.64 $\pm$ 0.07	1.00 $\pm$ 0.00	0.69 $\pm$ 0.06
W-DERIVED	1.00 $\pm$ 0.00	0.65 $\pm$ 0.07	1.00 $\pm$ 0.00	0.64 $\pm$ 0.07	0.47 $\pm$ 0.50	0.53 $\pm$ 0.08	0.85 $\pm$ 0.36	0.66 $\pm$ 0.10
W-DISTRIBUTE	1.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.63 $\pm$ 0.08	1.00 $\pm$ 0.00	0.61 $\pm$ 0.06	0.93 $\pm$ 0.25	0.66 $\pm$ 0.09
W-COND	0.93 $\pm$ 0.25	0.66 $\pm$ 0.10	0.87 $\pm$ 0.34	0.65 $\pm$ 0.10	0.40 $\pm$ 0.49	0.57 $\pm$ 0.12	0.60 $\pm$ 0.49	0.62 $\pm$ 0.13
W-MULTI	0.93 $\pm$ 0.25	0.66 $\pm$ 0.09	1.00 $\pm$ 0.00	0.70 $\pm$ 0.07	1.00 $\pm$ 0.00	0.65 $\pm$ 0.08	1.00 $\pm$ 0.00	0.69 $\pm$ 0.06
Average	0.96	0.68	0.97	0.69	0.55	0.53	0.78	0.63

Table 6: Workflow evaluation results for Photonics2D (mean $\pm$ std). TC = Task Completion rate, CO = Combined Overall score. **Bold** = best model per metric per row. Gray text shows Beams2D results on the same workflow styles for comparison.

Style	GPT-5-mini		Gemini-3-Flash		Qwen3-4B		Qwen3.5-4B	
	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$
W-RAND	1.00 $\pm$ 0.00	0.53 $\pm$ 0.01	1.00 $\pm$ 0.00	0.60 $\pm$ 0.02	1.00 $\pm$ 0.00	0.56 $\pm$ 0.02	1.00 $\pm$ 0.00	0.55 $\pm$ 0.01
W-DISTRIBUTE	1.00 $\pm$ 0.00	0.52 $\pm$ 0.02	1.00 $\pm$ 0.00	0.55 $\pm$ 0.03	1.00 $\pm$ 0.00	0.52 $\pm$ 0.02	1.00 $\pm$ 0.00	0.57 $\pm$ 0.05
W-COND	0.40 $\pm$ 0.49	0.47 $\pm$ 0.08	0.53 $\pm$ 0.50	0.54 $\pm$ 0.09	0.20 $\pm$ 0.40	0.43 $\pm$ 0.07	0.47 $\pm$ 0.50	0.49 $\pm$ 0.08
Average	0.80	0.51	0.84	0.56	0.73	0.50	0.82	0.54
<i>Beams 2D (same styles, for comparison)</i>								
W-RAND	1.00 $\pm$ 0.00	0.66 $\pm$ 0.07	1.00 $\pm$ 0.00	0.65 $\pm$ 0.09	1.00 $\pm$ 0.00	0.64 $\pm$ 0.07	1.00 $\pm$ 0.00	0.69 $\pm$ 0.06
W-DISTRIBUTE	1.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.63 $\pm$ 0.08	1.00 $\pm$ 0.00	0.61 $\pm$ 0.06	0.93 $\pm$ 0.25	0.66 $\pm$ 0.09
W-COND	0.93 $\pm$ 0.25	0.66 $\pm$ 0.10	0.87 $\pm$ 0.34	0.65 $\pm$ 0.10	0.40 $\pm$ 0.49	0.57 $\pm$ 0.12	0.60 $\pm$ 0.49	0.62 $\pm$ 0.13
Average	0.98	0.65	0.96	0.64	0.80	0.61	0.84	0.66

gap to proprietary models. Across all styles, W-COND is the hardest: the maximum task completion value is 0.93 for GPT-5-mini, as this style requires tool-output reasoning. Figure 2 shows a representative example: given identical inputs, Gemini-3-Flash selects the correct conditional branch and produces a design that passes parameter validation (IoU = 0.58), while Qwen3-4B fails parameter validation and yields a visibly different topology (IoU = 0.37) with larger error regions.

The combined overall score (CO) is comparable across GPT-5-mini and Gemini-3-Flash and drops for both open-source models, though Qwen3.5-4B (CO = 0.63 avg) substantially outperforms Qwen3-4B (CO = 0.53 avg). Notably, on W-COND, Qwen3-4B achieves a CO of 0.57 despite its low task completion (0.40), exceeding its CO on Full (0.45) and Natural (0.29) where it achieves zero task completion. The combined overall score can be nonzero even when task completion is zero, as the agent may still produce partial designs and valid tool calls that contribute to the design quality and tool-use components.

The apparent paradox of Qwen3-4B achieving TC = 0.00 on FULL and NATURAL but TC = 1.00 on W-RAND, W-DISTRIBUTE, and W-MULTI reflects differences in task completion criteria rather than general capability. FULL requires the agent to autonomously call *render_design* as the final step, which Qwen3-4B consistently omits; the W-styles instead provide explicit step-by-step instructions including the STL export call, compensating for the model’s weaker autonomous task sequencing. Similarly, on NATURAL the model calls optimization tools directly without detecting missing parameters and calling *ask_human_for_clarification*.

Qwen3.5-4B largely resolves the FULL deficit (TC = 0.73) but still performs poorly on NATURAL (TC = 0.33), suggesting that autonomous rendering has improved across model generations while detecting underspecified prompts remains challenging at this scale.

The tool-calling heatmaps (Figure 3) reveal model-specific patterns: on FULL, GPT-5-mini and Gemini-3-Flash invoke each expected tool exactly once, whereas Qwen3-4B makes redundant tool calls while omitting *render_design*—consistent with its zero task completion. On W-COND, Qwen3-4B again issues unnecessary calls, while

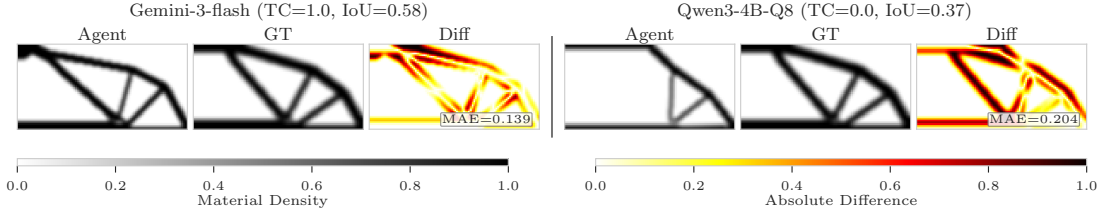


Figure 2: Design comparison for the W-COND style on the same problem instance (Beams2D, seed 3, example 3). Each group shows a different LLM backend: the agent-generated design (left), ground truth (center), and pixelwise absolute difference (right). Gemini-3-Flash selects the correct conditional branch and passes task completion (TC=1.0, IoU=0.58); Qwen3-4B fails parameter validation (TC=0.0, IoU=0.37), producing a visibly different topology with larger error regions.

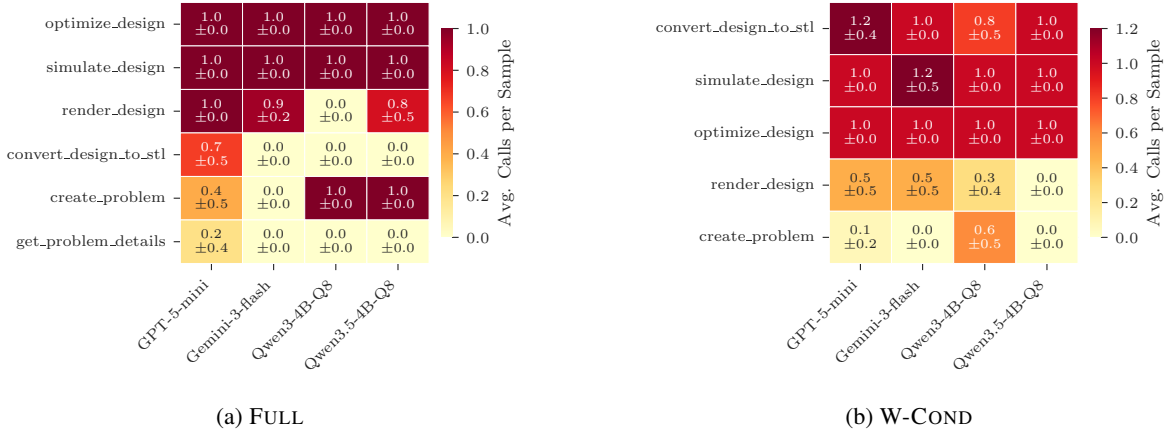


Figure 3: Tool-calling heatmaps for the FULL (a) and W-COND (b) prompt styles. Each cell shows the average number of calls per tool across all samples. FULL shows consistent tool usage across models; W-COND reveals divergent patterns for the open-source models.

Qwen3.5-4B achieves optimal efficiency by calling each tool exactly once. The combined overall score distributions (Figure 4) corroborate these findings: proprietary models cluster tightly near $CO \approx 0.7$ on FULL, while the open-source models show broader, lower-scoring distributions that widen further on W-COND. Figure 5 further shows that on W-RAND, excess tool calls degrade CO without improving design quality, which remains flat at ≈ 0.5 ; we return to this in the discussion of tool usage efficiency (Section 5).

4.1.1 Photonics2D Generalization

Table 6 evaluates all four backends on Photonics2D using three prompt styles. Before discussing model differences, we note that on Photonics2D several design-quality sub-scores are near-constant by construction: pixel accuracy is inflated by background, constraint match is trivially enforced by the optimizer, and 2D connectivity fails uniformly on the multi-component photonic designs. The discriminative signal therefore comes primarily from task completion and tool efficiency, which we report as the headline metric below. On W-RAND and W-DISTRIBUTE, all models achieve perfect task completion (TC=1.00), mirroring Beams2D. However, W-COND proves substantially harder: even the best model (Gemini-3-Flash) reaches only 53% TC, compared to 87% on Beams2D, with Qwen3.5-4B at 47%, GPT-5-mini at 40%, and Qwen3-4B at 20%. Notably, the W-COND failures are not caused by missing tool calls—all models invoke *convert_design_to_stl* in 100% of runs—but by *branch inversion*: in all 36 failed runs across four models, the agent selects the exact opposite conditional branch, swapping the branch-conditional parameters (*threshold*, *mirror_y*) while leaving common parameters (*scale_xy*, *scale_z*) correct. This rules out random errors and indicates that agents parse the conditional structure but invert the comparison (see Appendix D.2 for tool-calling heatmaps and score distributions). A likely contributing factor is the nature of the objective: *compliance* in Beams2D is a standard structural mechanics quantity, whereas *total_overlap* in Photonics2D is a domain-specific metric less likely to appear in pretraining corpora, making the threshold comparison less intuitive.

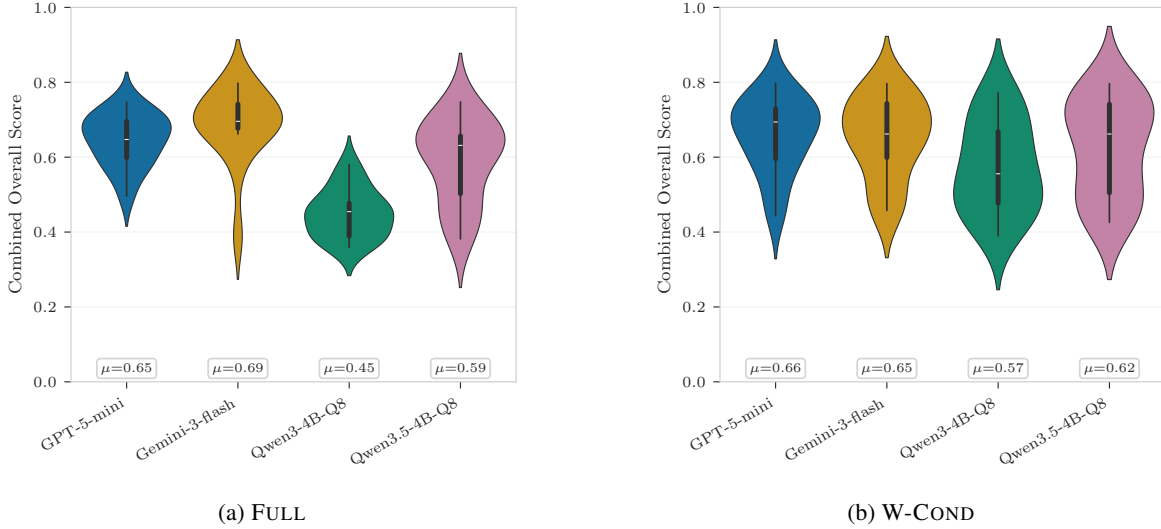


Figure 4: Combined overall score distributions for the FULL (a) and W-COND (b) prompt styles. FULL establishes baseline capabilities; W-COND reveals degradation for open-source models due to conditional reasoning demands.

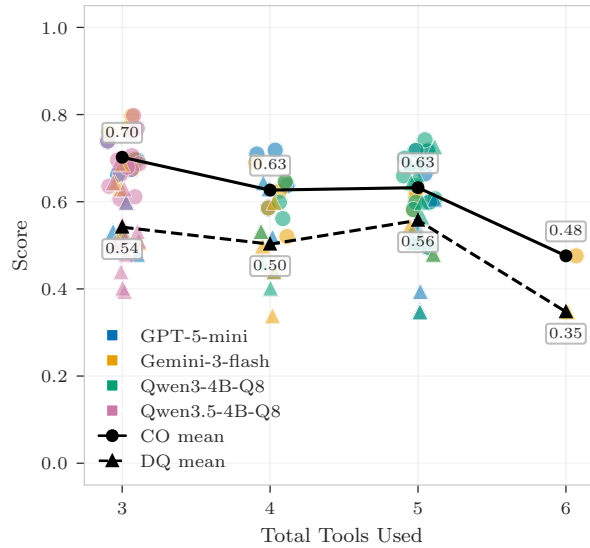


Figure 5: Tool count vs. performance for the W-RAND style. The solid line shows the combined overall score (CO) declining with additional tool calls, while the dashed line shows the design quality score (DQ) remaining flat (≈ 0.5), indicating that extra calls penalize efficiency without improving engineering output.

4.2 RAG Evaluation

Having established workflow performance across prompt styles and models, we next evaluate whether document retrieval improves the agent’s engineering parameter selection.

Figure 6 shows the weighted score contributions for the four prompts evaluated on three LLM backends (GPT-5-mini, Gemini-3-Flash, and Qwen3-4B)³ under the three conditions defined in Section 3.4.2, each averaged over three runs. RAG-off trivially yields scores of exactly 0 due to the gated scoring mechanism and is omitted from the figure. RAG-on approaches a score of 1.0, while Empty RAG degrades substantially, confirming that agent accuracy depends on retrieved document content rather than prior knowledge.

³Qwen3.5-4B was excluded from the RAG and HPC evaluations as it was added after these experiments were completed.

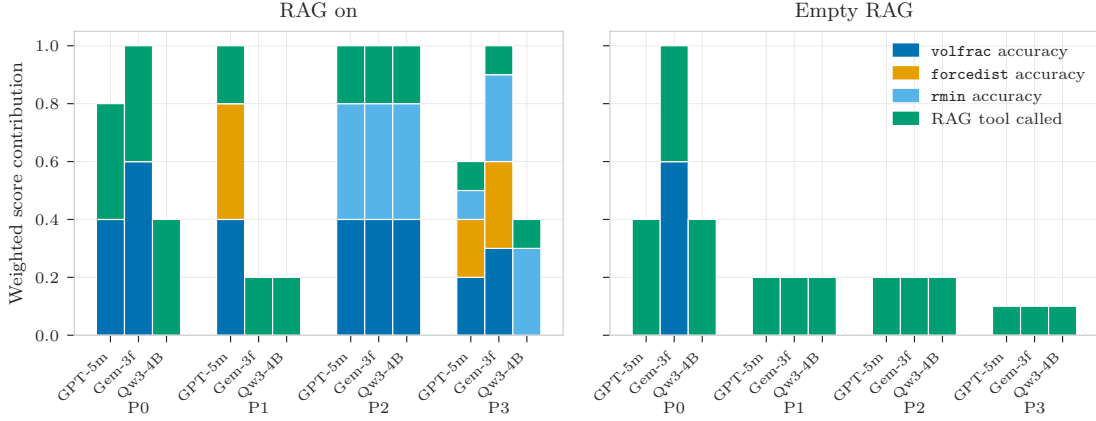


Figure 6: Weighted RAG score contributions by prompt and LLM backend under RAG-on and Empty RAG conditions (3 runs each). RAG-off (all scores exactly 0) is omitted. RAG-on approaches 1.0 for most combinations; Empty RAG degrades substantially except for Gemini on P0, where the default volume fraction is likely memorized.

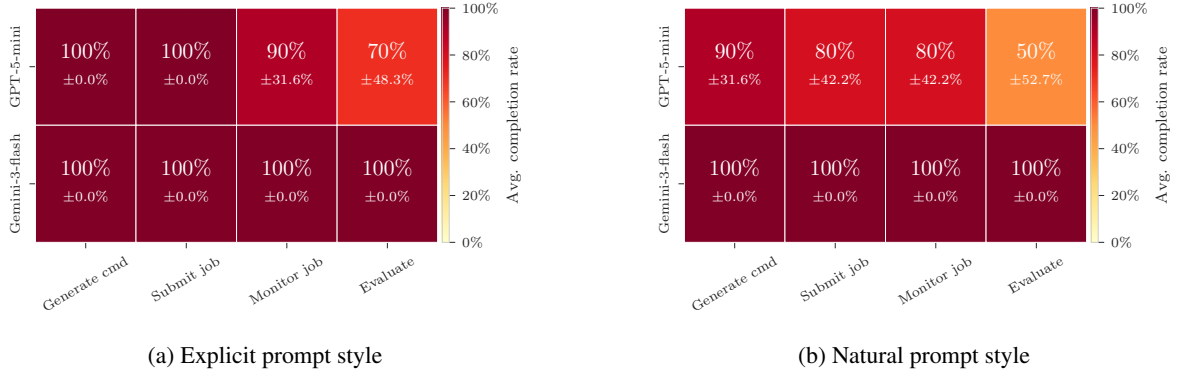


Figure 7: Average step completion rates for the cGAN HPC training benchmark. (a) Explicit: step-by-step tool instructions. (b) Natural: plain-language description. Each cell shows the mean fraction of runs completing that step, averaged across 10 seeds.

For prompt 0 (P0), Gemini achieves a high score even with an empty index. A likely explanation is that P0 asks for a volume fraction of 0.35, a widely used value in topology optimization (e.g., Sigmund’s 99-line benchmark [44]) that also serves as the EngiBench default — making it plausibly memorized in the model’s training data. In contrast, P2 and P3 target parameters from a paper published after the models’ training data cutoff (arXiv:2505.02438), which no model can produce without retrieval. Different models perform best on different prompts: for P0 and P3, Gemini performs better than GPT, while the opposite holds for P1. The Qwen3-4B scores are generally lower except for P2, where all models achieve the same score.

4.3 HPC Training Evaluation

The final evaluation dimension tests whether the multi-agent system scales to long-running, compute-intensive training pipelines on remote infrastructure. We restrict the HPC evaluation to the two proprietary models due to the substantial wall-clock time each run requires (remote job submission, training, and monitoring).

Figure 7 compares step completion rates under two prompt styles: the explicit style provides step-by-step tool-calling instructions, while the natural style describes the same workflow in plain language. Gemini-3-Flash achieves 100% completion across all steps under both prompt styles. GPT-5-mini, however, degrades progressively: under explicit instructions it reaches 70% at the final evaluation step, while under natural-language instructions this drops further to 50%, with earlier steps also affected (90% → 80% for job submission and monitoring).

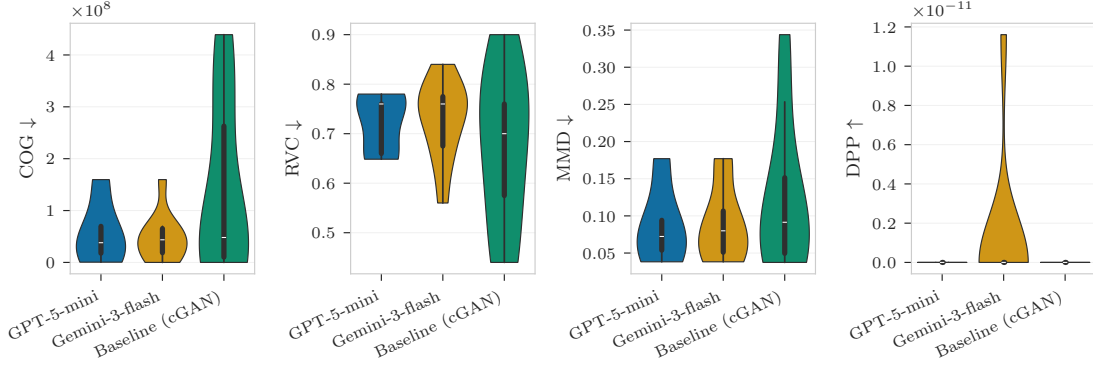


Figure 8: Offline model quality metrics (COG, RVC, MMD, DPP) for agent-trained cGAN models vs. EngiBench baselines. Arrows indicate desired direction. Values averaged across available seeds.

The root cause is multi-step instruction degradation: GPT-5-mini reliably executes initial steps but inconsistently follows through on later ones—most commonly skipping the final *evaluate_model* call. These are not timeout or tool-error failures; the agent simply ceases to issue subsequent calls. The drop from explicit to natural instructions amplifies this effect, indicating that GPT-5-mini relies on explicit step enumeration to maintain sequencing fidelity over long-running workflows. By contrast, Gemini-3-Flash is robust to instruction style, completing all four steps in 100% of runs.

Figure 8 compares the offline model quality metrics for the cGAN algorithm. Agent-trained models achieve comparable values to the EngiBench baselines, with minor variations attributable to non-deterministic execution on heterogeneous HPC nodes.

5 Discussion

5.1 Key Findings

Our evaluation yields the following answers to the research questions posed in Section 1:

RQ1 (Workflow Performance): GPT-5-mini and Gemini-3-Flash achieve 96–97% average task completion across all seven Beams2D prompt styles (Table 5), indicating that current-generation LLMs can reliably execute structured engineering tool chains when given explicit instructions. The open-source Qwen3.5-4B reaches 78%, a considerable improvement over Qwen3-4B (55%), demonstrating rapid generational progress at fixed model scale. On Photonics2D (Table 6), W-COND proves considerably harder (53% for the best model), suggesting that conditional reasoning difficulty is problem-dependent and tied to domain familiarity.

RQ2 (Model Robustness): Performance is consistent between GPT-5-mini and Gemini-3-Flash. The two open-source 4B-parameter models show that generational improvements can partially compensate for scale: Qwen3.5-4B closes the gap on workflow styles where Qwen3-4B failed (e.g., FULL TC: 0.00 $\rightarrow$ 0.73, W-DERIVED TC: 0.47 $\rightarrow$ 0.85), though both still underperform proprietary models on the hardest styles. This generational trend extends to Photonics2D, where Qwen3.5-4B more than doubles its predecessor’s W-COND task completion (0.20 $\rightarrow$ 0.47).

RQ3 (Tool Usage Efficiency): Tool usage analysis reveals that all models call the correct tools in the expected order for standard workflows, but natural and conditional prompts expose divergent tool-calling patterns. The tool-count analysis (Figure 5) shows that excess tool calls degrade the combined overall score from 0.70 (3 tools) to 0.63 (4–5 tools) without improving design quality, which remains flat at ≈ 0.5 . This demonstrates that tool efficiency directly impacts the combined score through its efficiency component, while the engineering output quality is robust to over-calling. Of note, Qwen3.5-4B achieves optimal tool efficiency on several workflow styles by calling exactly the required tools with no unnecessary invocations, explaining its competitive combined scores despite lower task completion.

RQ4 (RAG Augmentation): RAG-on runs approach a score of 1.0 on most model–prompt combinations (Figure 6), while RAG-off runs score exactly 0 due to the gated scoring mechanism. The Empty RAG condition degrades substantially, confirming that agent accuracy depends on the content returned by the retrieval system, not merely on the presence of the retrieval tool. This validates the gated scoring design: without retrieval, even correct parameter guesses receive no credit, ensuring the benchmark measures retrieval-augmented decision-making specifically.

RQ5 (HPC Orchestration): Within the two cloud models evaluated (Gemini-3-Flash and GPT-5-mini), the HPC agent successfully orchestrates end-to-end ML training pipelines on SLURM-based clusters (Figure 7), including job submission, monitoring, and result retrieval. However, success is model-dependent: Gemini-3-Flash completes the full pipeline in 100% of runs, while GPT-5-mini drops to 70% under explicit instructions and 50% under natural-language prompts at the final evaluation step. This suggests that for the tested proprietary models, multi-step instruction following degrades over long-running workflows, even for models that perform well on shorter tool chains.

5.2 Comparison to Prior Work

As established in Section 2 (Table 1), no existing system simultaneously covers all six capability dimensions. We compare our results to the most relevant prior work along each evaluation dimension.

Workflow Performance The closest multi-agent engineering system is the 9-agent MAS with Design State Graph (DSG) coordination [21]. Direct numerical comparison is not meaningful because the two systems address structurally different tasks: DSG-MAS targets full design-state management across coupled requirement–design–test loops, whereas our benchmark evaluates tool-calling workflows on structured engineering pipelines. We therefore position DSG-MAS as a complementary contribution rather than a numerical baseline; matched-task comparison would require a shared evaluation framework that does not currently exist.

Our workflow benchmark also extends the scope of existing engineering evaluations. FDM-Bench [37] evaluates LLM on G-code anomaly detection and user queries but does not test multi-step tool orchestration. General-purpose tool-use benchmarks such as ToolSandbox [31] and ACEBench [30] report significant performance gaps between open-source and proprietary models, consistent with the degradation we observe for the open-source 4B-parameter models on complex prompt styles, though generational improvements (Qwen3-4B $\rightarrow$ Qwen3.5-4B) can partially close this gap. Similarly, τ -bench [39] reports $<50\%$ task success (pass⁸ $< 25\%$) for multi-turn professional tasks; our higher completion rates likely reflect the more constrained, tool-bounded nature of engineering workflows compared to open-ended customer service scenarios.

RAG Evaluation While RAG is a well-established technique [25, 26], its evaluation in engineering contexts has focused on document comprehension rather than actionable parameter extraction. Doris *et al.* [49] benchmark multimodal LLM understanding of engineering documentation (rule comprehension, compliance, and extraction), but do not evaluate whether retrieved information is correctly applied to downstream tool calls. Our gated scoring mechanism addresses this gap by isolating the contribution of document retrieval to parameter selection, revealing that agents can reliably extract domain-specific parameters when documents are indexed but cannot produce post-cutoff values without retrieval.

HPC Orchestration The closest work in automating simulation execution is ALL-FEM [16], which uses multi-agent LLM collaboration to generate and execute FEniCS code on Google Colab, achieving 71.79% accuracy across 39 FEA benchmarks. However, ALL-FEA targets code generation for simulation execution in a notebook environment rather than HPC job orchestration. To the best of our knowledge, no prior work benchmarks end-to-end LLM-driven HPC training for engineering ML models. The ability to generate Simple Linux Utility for Resource Management (SLURM) scripts, submit jobs to a production cluster, monitor execution, and evaluate trained models—achieving metrics comparable to official EngiBench baselines—demonstrates a capability not addressed by existing agent benchmarks.

General-Purpose Coding Agents General-purpose coding agents—such as OpenAI Codex [50], Claude Code [51], and OpenClaw [52]—can write and execute arbitrary code, potentially calling EngiBench and EngiOpt APIs directly without pre-bound tool abstractions. Our benchmark suite is agent-agnostic: any system capable of invoking the underlying Python APIs—whether through pre-bound tools or generated code—can be evaluated on the same prompt styles and scoring methodology with minimal adaptation.

5.3 Limitations and Threats to Validity

The current implementation and evaluation have several limitations. First, the benchmark problems are limited to Beams2D and Photonics2D and do not cover the full range of EngiBench problems. Second, a user study with practicing engineers who could intervene to re-direct the agents was omitted due to time constraints and the cost of recruiting domain experts, though this could be a promising direction of future work. Third, only four LLM backends were tested, primarily due to API cost constraints; the rapid pace of model releases means that newer models may alter the relative performance rankings reported here. Finally, the HPC training benchmark covers only two generative models (cGAN and diffusion) and two cloud LLM backends (Gemini-3-Flash and GPT-5-mini), reflecting the high per-model

evaluation cost and the prohibitively slow local inference of the open-source 4B-parameter models on our available hardware for the long-horizon HPC workflow; RQ5 conclusions are accordingly scoped to current proprietary LLMs on this specific cluster and training setup. We also do not include a single-agent baseline with identical tool access against the multi-agent supervisor; an empirical ablation of the supervisor decomposition lies outside the scope of the present benchmark study. The present analysis is descriptive: we report means and standard deviations but do not provide statistical significance testing, confidence intervals, or sensitivity analyses across composite weights, prompt phrasings, or temperature settings.

Regarding threats to validity: despite setting temperature = 0, LLM API responses exhibit minor run-to-run variation due to floating-point non-determinism and provider-side batching. We mitigate this by averaging over multiple seeds; the resulting standard deviations (Tables 5–6) are small relative to the performance differences between models, indicating that this noise does not affect the conclusions.

6 Conclusion and Future Work

In this paper, we introduced a benchmark suite for evaluating LLM-driven engineering workflows along three dimensions—workflow execution under distinct cognitive demands, gated retrieval-augmented parameter selection, and HPC training orchestration—together with ENGIAI, a multi-agent reference implementation that operationalizes the benchmark. The system coordinates seven specialized agents—covering topology optimization, ML-based design generation, document retrieval, web search, HPC job management, local command execution, and 3D printer control—through a supervisor architecture built on LangChain and LangGraph.

Our evaluation across three benchmark dimensions and four LLM backends yielded several key findings. On workflow tasks, proprietary models (GPT-5-mini and Gemini-3-Flash) achieved 96–97% average task completion across seven prompt styles targeting distinct cognitive demands, while the open-source 4B-parameter models reached 55% (Qwen3-4B) and 78% (Qwen3.5-4B), demonstrating rapid generational improvement at fixed model scale. The gated RAG scoring mechanism confirmed that RAG is essential for parameter selection from post-cutoff literature, with RAG-on scores approaching 1.0 and near-zero scores without retrieval. The HPC training benchmark demonstrated that the two tested cloud models can autonomously orchestrate end-to-end model training on SLURM clusters, achieving metrics comparable to official baselines.

These results indicate that LLM-based multi-agent systems can reliably support structured engineering workflows when backed by sufficiently capable foundation models.

Several directions follow naturally from the unresolved aspects of our research questions.

Broader workflow and model coverage (RQ1–RQ3) Our evaluation covers two physics domains, four models, and two open-source generations from the same family. Extending to additional EngiBench problems, larger open-source models (e.g., 70B parameter), and other model families (e.g., Llama, Mistral) would clarify whether the prompt-style failure-mode and tool over-calling patterns we observe are scale-dependent, family-specific, or general. Sensitivity analysis—varying temperature, prompt phrasing, or tool description verbosity—would further quantify the fragility of current performance levels. On tool efficiency specifically, it remains open whether tool-use patterns transfer across problems and whether techniques such as tool-use demonstrations or constrained decoding can reduce unnecessary invocations for smaller models. Beyond coverage, the evaluation exposes concrete failure patterns that suggest targeted interventions. The tool over-calling observed in smaller models—where excess invocations degrade the combined score from 0.70 to 0.63 without improving design quality (Section 4)—could be mitigated through few-shot tool-use demonstrations in the system prompt or constrained decoding that restricts tool selection based on the current workflow state. The conditional-reasoning failures on W-COND, particularly for Photonics2D where even the best model reaches only 53%, point to a need for structured chain-of-thought: requiring the agent to explicitly extract the simulation result, state the comparison, and declare the chosen branch before issuing the tool call. The semantic-disambiguation failures on W-DISTRIBUTE—where models select preview-context values instead of export-context values—suggest that parameter extraction should be decoupled from tool invocation, for instance by first generating a structured parameter plan that can be validated before execution. More broadly, the benchmark itself generates structured tool-call traces with ground-truth validation, which could serve as training data for domain-specific fine-tuning of smaller open-source models—using the scorer’s pass/fail signal as a reward to directly optimize tool-calling accuracy in engineering workflows.

Scaling retrieval and HPC orchestration (RQ4–RQ5) Our RAG evaluation uses a curated index of two papers; testing with larger, noisier collections where relevant information must be identified among contradictory sources would better approximate real engineering practice. Scaling the index to tens or hundreds of papers would test retrieval precision under realistic conditions where relevant information is sparse relative to the corpus size. A further step is

adversarial retrieval evaluation: indexing documents that contain plausible but incorrect parameter values alongside the correct source, testing whether the agent can resolve conflicting evidence—analogue to the W-DISTRIBUTE style but at the retrieval level rather than the prompt level. Retrieval quality itself could also be improved through domain-specific chunking strategies (e.g., preserving table and equation boundaries) or re-ranking models fine-tuned on engineering literature. For HPC, a more demanding test is whether the agent can autonomously write and iterate on training code rather than orchestrating predefined scripts—shifting the evaluation from workflow orchestration to code generation and self-correction. The degradation observed for GPT-5-mini on long-running HPC pipelines (dropping to 50% on natural-language prompts at the final evaluation step) suggests that explicit state tracking—e.g., a structured checkpoint that records which pipeline stages have completed and their outputs—could help maintain instruction adherence across extended multi-step workflows. Finally, as the tool ecosystem grows via MCP [22] or tool APIs, a key open question is how agent performance scales with the number of available tools.

References

- [1] Wynn, David C and Eckert, Claudia M. “Perspectives on iteration in design and development.” *Research in Engineering Design* Vol. 28 No. 2 (2017): pp. 153–184.
- [2] Regenwetter, Lyle, Nobari, Amin Heyrani and Ahmed, Faez. “Deep generative models in engineering design: A review.” *Journal of Mechanical Design* Vol. 144 No. 7 (2022): p. 071704.
- [3] OpenAI. “ChatGPT [Large language model].” <https://chat.openai.com> (2026).
- [4] LangChain, Inc. “LangGraph: Build Resilient Language Agents as Graphs.” (2024). URL <https://github.com/langchain-ai/langgraph>. Open-source Python library.
- [5] Beitz, W, Pahl, G and Grote, K. “Engineering design: a systematic approach.” *Mrs Bulletin* Vol. 71 No. 30 (1996): p. 3.
- [6] Felten, Florian, Apaza, Gabriel, Bräunlich, Gerhard et al. “EngiBench: A Framework for Data-Driven Engineering Design Research.” *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2025. URL <https://openreview.net/forum?id=YowD33Q89V>.
- [7] Jiang, Shuo, Xie, Min, Chen, Frank Youhua et al. “Intelligent Design 4.0: Paradigm Evolution Toward the Agentic Artificial Intelligence Era.” *Journal of Computing and Information Science in Engineering* Vol. 25 No. 12 (2025): p. 120808. doi:10.1115/1.4070438. URL <https://asmedigitalcollection.asme.org/computingengineering/article-pdf/25/12/120808/7569711/jcise-25-1279.pdf>, URL <https://doi.org/10.1115/1.4070438>.
- [8] Acharya, Deepak Bhaskar, Kuppan, Karthigeyan and Divya, B. “Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey.” *IEEE Access* Vol. 13 (2025): pp. 18912–18936. doi:10.1109/ACCESS.2025.3532853.
- [9] Gridach, Mourad, Nanavati, Jay, Mack, Christina et al. “Agentic AI for Scientific Discovery: A Survey of Progress, Challenges, and Future Directions.” *Towards Agentic AI for Science: Hypothesis Generation, Comprehension, Quantification, and Validation*. 2025. URL <https://openreview.net/forum?id=TyCYakX9BD>.
- [10] Wu, Qingyun, Bansal, Gagan, Zhang, Jieyu et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” *First Conference on Language Modeling (COLM)*. 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>. ArXiv:2308.08155.
- [11] CrewAI. “CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents.” (2024). URL <https://github.com/crewAIInc/crewAI>. Open-source Python library.
- [12] OpenAI. “OpenAI Agents SDK.” (2025). URL <https://github.com/openai/openai-agents-python>. Open-source Python library.
- [13] Gottweis, Juraj, Weng, Wei-Hung, Daryin, Alexander et al. “Towards an AI co-scientist.” (2025). doi:10.48550/arXiv.2502.18864. URL <http://arxiv.org/abs/2502.18864>. ArXiv:2502.18864 [cs].
- [14] Ni, Bo and Buehler, Markus J. “MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge.” *Extreme Mechanics Letters* Vol. 67 (2024): p. 102131. doi:<https://doi.org/10.1016/j.eml.2024.102131>. URL <https://www.sciencedirect.com/science/article/pii/S2352431624000117>.
- [15] Qi, Yupeng, Xu, Ran and Chu, Xu. “FeaGPT: an End-to-End agentic-AI for Finite Element Analysis.” (2025). doi:10.48550/arXiv.2510.21993. URL <https://arxiv.org/abs/2510.21993>.

- [16] Deotale, Rushikesh, Srinivasan, Adithya, Tian, Yuan et al. “ALL-FEM: Agentic Large Language Models Fine-Tuned for Finite Element Methods.” *SSRN Electronic Journal* (2026)doi:10.2139/ssrn.6103826. URL <https://ssrn.com/abstract=6103826>.
- [17] Pradas-Gomez, Alejandro, Brahma, Arindam and Isaksson, Ola. “DUCTILE: Agentic LLM Orchestration of Engineering Analysis in Product Development Practice.” (2026). URL 2603.10249, URL <https://arxiv.org/abs/2603.10249>.
- [18] Elrefaie, Mohamed, Qian, Janet, Wu, Raina et al. “AI Agents in Engineering Design: A Multi-Agent Framework for Aesthetic and Aerodynamic Car Design.” *Volume 3B: 51st Design Automation Conference (DAC)*. 2025. American Society of Mechanical Engineers. doi:10.1115/detc2025-169682. URL <http://dx.doi.org/10.1115/DETC2025-169682>.
- [19] Chen, Pei, Cai, Yichen, Zhou, Zihong et al. “An LLM-based multi-agent system to assist early-stage product design and evaluation.” *Journal of Engineering Design* Vol. 37 No. 3 (2026): pp. 945–980. doi:10.1080/09544828.2026.2616583. URL <https://doi.org/10.1080/09544828.2026.2616583>, URL <https://doi.org/10.1080/09544828.2026.2616583>.
- [20] Wang, Zeyu, Lo, Frank Po Wen, Chen, Qian et al. “An LLM-enabled multi-agent autonomous mechatronics design framework.” *Proceedings of the computer vision and pattern recognition conference*: pp. 4205–4215. 2025.
- [21] Massoudi, Soheyl and Fuge, Mark. “Agentic Large Language Models for Conceptual Systems Engineering and Design.” *Journal of Mechanical Design* Vol. 148 No. 5 (2026): p. 051405. doi:10.1115/1.4070328. URL <https://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/148/5/051405/7561928/md-25-1500.pdf>, URL <https://doi.org/10.1115/1.4070328>.
- [22] Anthropic. “Model Context Protocol.” (2025). URL <https://modelcontextprotocol.io/specification/2025-11-25>.
- [23] Wang, Zhenting, Chang, Qi, Patel, Hemani et al. “MCP-Bench: A Benchmark for Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers.” (2025). doi:10.48550/arXiv.2504.11457. ArXiv:2504.11457.
- [24] Jadhav, Yayati, Pak, Peter and Barati Farimani, Amir. “LLM-3D print: Large Language Models to monitor and control 3D printing.” *Additive Manufacturing* Vol. 114 (2025): p. 105027. doi:<https://doi.org/10.1016/j.addma.2025.105027>. URL <https://www.sciencedirect.com/science/article/pii/S2214860425003926>.
- [25] Lewis, Patrick, Perez, Ethan, Piktus, Aleksandra et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks.” *Proceedings of the 34th International Conference on Neural Information Processing Systems*. 2020. Curran Associates Inc., Red Hook, NY, USA.
- [26] Gao, Yunfan, Xiong, Yun, Gao, Xinyu et al. “Retrieval-Augmented Generation for Large Language Models: A Survey.” (2024). doi:10.48550/arXiv.2312.10997. URL <http://arxiv.org/abs/2312.10997>. ArXiv:2312.10997 [cs].
- [27] Chandrasekhar, Achuth, Chan, Jonathan, Ogoke, Francis et al. “AMGPT: A large language model for contextual querying in additive manufacturing.” *Additive Manufacturing Letters* Vol. 11 (2024): p. 100232. doi:<https://doi.org/10.1016/j.addlet.2024.100232>. URL <https://www.sciencedirect.com/science/article/pii/S2772369024000409>.
- [28] Khanghah, Kiarash Naghavi, Chen, Zhiling, Romeo, Lela et al. “Zero-Shot Anomaly Detection in Laser Powder Bed Fusion Using Multimodal Retrieval-Augmented Generation and Large Language Models.” *Journal of Mechanical Design* Vol. 148 No. 7 (2025): p. 072001. doi:10.1115/1.4070585. URL <https://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/148/7/072001/7573644/md-25-1625.pdf>, URL <https://doi.org/10.1115/1.4070585>.
- [29] Mohammadi, Mahmoud, Li, Yipeng, Lo, Jane et al. “Evaluation and Benchmarking of LLM Agents: A Survey.” *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*: p. 6129–6139. 2025. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3711896.3736570. URL <https://doi.org/10.1145/3711896.3736570>.
- [30] Chen, Chen, Hao, Xinlong, Liu, Weiwen, Huang, Xu, Zeng, Xingshan, Yu, Shuai, Li, Dexun, Huang, Yuefeng, Liu, Xiangcheng, Xinzhi, Wang and Liu, Wu. “ACEBench: A Comprehensive Evaluation of LLM Tool Usage.” Christodoulopoulos, Christos, Chakraborty, Tanmoy, Rose, Carolyn and Peng, Violet (eds.). *Findings of the Association for Computational Linguistics: EMNLP 2025*: pp. 12970–12998. 2025. Association for Computational Linguistics, Suzhou, China. doi:10.18653/v1/2025.findings-emnlp.697. URL <https://aclanthology.org/2025.findings-emnlp.697/>.

- [31] Lu, Jiarui, Holleis, Thomas, Zhang, Yizhe et al. “ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities.” Chiruzzo, Luis, Ritter, Alan and Wang, Lu (eds.). *Findings of the Association for Computational Linguistics: NAACL 2025*: pp. 1160–1183. 2025. Association for Computational Linguistics, Albuquerque, New Mexico. doi:10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [32] Patil, Shishir G, Mao, Huanzhi, Yan, Fanjia et al. “The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models.” *Forty-second International Conference on Machine Learning*. 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>.
- [33] Qin, Yujia, Liang, Shihao, Ye, Yining et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs.” *The Twelfth International Conference on Learning Representations*. 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- [34] Liu, Xiao, Yu, Hao, Zhang, Hanchen et al. “AgentBench: Evaluating LLMs as Agents.” *The Twelfth International Conference on Learning Representations*. 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- [35] Ma, Chang, Zhang, Junlei, Zhu, Zhihao et al. “AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents.” (2024). doi:10.48550/arXiv.2401.13178. URL <http://arxiv.org/abs/2401.13178>. ArXiv:2401.13178 [cs].
- [36] Chen, Zirui, Chen, Shijie, Ning, Yuting et al. “ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery.” *The Thirteenth International Conference on Learning Representations*. 2025. URL <https://openreview.net/forum?id=6z4YKr0GK6>.
- [37] Eslaminia, Ahmadreza, Jackson, Adrian, Tian, Beitong et al. “FDM-bench: a domain-specific benchmark for evaluating large language models in additive manufacturing.” *Manufacturing Letters* Vol. 44 (2025): pp. 1415–1424. doi:<https://doi.org/10.1016/j.mfglet.2025.06.161>. URL <https://www.sciencedirect.com/science/article/pii/S2213846325001968>. 53rd SME North American Manufacturing Research Conference (NAMRC 53).
- [38] Zhou, Xiyuan, Wang, Xinlei, He, Yirui et al. “EngiBench: A Benchmark for Evaluating Large Language Models on Engineering Problem Solving.” (2025). doi:10.48550/arXiv.2509.17677. URL <http://arxiv.org/abs/2509.17677>. ArXiv:2509.17677 [cs].
- [39] Yao, Shunyu, Shinn, Noah, Razavi, Pedram et al. “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.” *The Thirteenth International Conference on Learning Representations*. 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- [40] He, Hongliang, Yao, Wenlin, Ma, Kaixin et al. “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models.” *arXiv preprint arXiv:2401.13919* (2024).
- [41] Deng, Xiang, Gu, Yu, Zheng, Boyuan et al. “Mind2Web: Towards a Generalist Agent for the Web.” (2023). doi:10.48550/arXiv.2306.06070. URL <http://arxiv.org/abs/2306.06070>. ArXiv:2306.06070 [cs].
- [42] Chase, Harrison. “LangChain.” (2022). URL <https://github.com/langchain-ai/langchain>.
- [43] Sallinen, Alexandre, Krsteski, Stefan, Teiletche, Paul et al. “M(M)ORE : Massive Multimodal Open RAG & Extraction.” *Championing Open-source DEvelopment in ML Workshop @ ICML25*. 2025. URL <https://openreview.net/forum?id=6j1HjfIdKn>.
- [44] Sigmund, O. “A 99 line topology optimization code written in Matlab.” *Struct. Multidiscip. Optim.* Vol. 21 No. 2 (2001): p. 120–127. doi:10.1007/s001580050176. URL <https://doi.org/10.1007/s001580050176>.
- [45] Andreassen, Erik, Clausen, Anders, Schevenels, Mattias et al. “Efficient topology optimization in MATLAB using 88 lines of code.” *Structural and Multidisciplinary Optimization* Vol. 43 No. 1 (2011): pp. 1–16. doi:10.1007/s00158-010-0594-7. URL <http://link.springer.com/10.1007/s00158-010-0594-7>.
- [46] Singh, Aaditya, Fry, Adam, Perelman, Adam et al. “OpenAI GPT-5 System Card.” (2025). URL 2601.03267, URL <https://arxiv.org/abs/2601.03267>.
- [47] Yang, An, Li, Anfeng, Yang, Baosong et al. “Qwen3 Technical Report.” (2025). doi:10.48550/arXiv.2505.09388. URL <http://arxiv.org/abs/2505.09388>. ArXiv:2505.09388 [cs].
- [48] He, Liang, Wei, Huayi and Tian, Tian. “SOPTX: A High-Performance Multi-Backend Framework for Topology Optimization.” (2025). doi:10.48550/arXiv.2505.02438. URL <http://arxiv.org/abs/2505.02438>. ArXiv:2505.02438 [math].
- [49] Doris, Anna C., Grandi, Daniele, Tomich, Ryan et al. “DesignQA: A Multimodal Benchmark for Evaluating Large Language Models’ Understanding of Engineering Documentation.” *Journal of Computing and Information Science in Engineering* Vol. 25 No. 2 (2024): p. 021009.

doi:10.1115/1.4067333. URL https://asmedigitalcollection.asme.org/computingengineering/article-pdf/25/2/021009/7419451/jcise_25_2_021009.pdf, URL <https://doi.org/10.1115/1.4067333>.

- [50] OpenAI. “Codex.” (2025). URL <https://openai.com/codex/>.
- [51] Anthropic. “Claude Code.” (2025). URL <https://docs.anthropic.com/en/docs/claude-code>.
- [52] OpenClaw Contributors. “OpenClaw: An Open-Source Agentic Coding Framework.” (2025). URL <https://github.com/openclaw/openclaw>.
- [53] Saaty, R.W. “The analytic hierarchy process—what it is and how it is used.” *Mathematical Modelling* Vol. 9 No. 3 (1987): pp. 161–176. doi:[https://doi.org/10.1016/0270-0255\(87\)90473-8](https://doi.org/10.1016/0270-0255(87)90473-8). URL <https://www.sciencedirect.com/science/article/pii/0270025587904738>.

A Scoring Methodology

A.1 Design Quality Metrics

The design quality score is a weighted combination of six per-design metrics:

$$S_{\text{design}} = 0.31 \cdot \text{IoU} + 0.19 \cdot \text{PA} + 0.15 \cdot \text{Obj} + 0.12 \cdot \text{Constr} + 0.12 \cdot \text{Conn} + 0.11 \cdot \text{WT} \quad (1)$$

where:

- **IoU** (Intersection over Union): overlap between the binarized agent design and ground-truth design, $\text{IoU} = |A \cap G| / |A \cup G|$, with threshold 0.5.
- **PA** (Pixel Accuracy): fraction of correctly classified pixels, $\text{PA} = \frac{1}{N} \sum_i \mathbf{1}(A_i = G_i)$, where N is the total number of pixels.
- **Obj** (Objective Score): normalized score based on problem-specific objectives (e.g., compliance, field overlap).
- **Constr** (Constraint Score): smooth partial credit for constraint satisfaction, $\exp(-|v_{\text{actual}} - v_{\text{target}}|/\tau)$ with temperature τ , averaged across all constraints.
- **Conn** (2D Connectivity): validates topological connectivity of the 2D design.
- **WT** (3D Watertightness): validates that the exported 3D mesh forms a closed, watertight solid. In practice, WT is uniformly 0.00 across all configurations because the threshold-and-extrude STL export produces non-manifold meshes; a post-processing repair step would be needed for watertight output.

Weights were assigned by engineering judgment, prioritizing geometric fidelity over engineering performance metrics and printability. A post-hoc Analytic Hierarchy Process (AHP) analysis [53] confirmed consistency (CR = 0.012) and produced weights within ± 0.03 of the chosen values.

The hierarchical workflow score combines design quality with operational metrics:

$$S_{\text{workflow}} = 0.65 \cdot S_{\text{design}} + 0.20 \cdot S_{\text{tool}} + 0.15 \cdot S_{\text{completion}} \quad (2)$$

where S_{tool} is the tool-call efficiency, defined as the ratio of correctly matched calls to the maximum of optimal and actual call counts, and $S_{\text{completion}} \in \{0, 1\}$ indicates whether all required tools were called successfully. For styles with STL export (W-RAND through W-MULTI), task completion additionally requires that the export parameters match the prompt specifications within a tolerance of ± 0.05 for floats and exact match for booleans; a successful tool call with incorrect parameters counts as failed. For the NATURAL style, when the agent correctly requests clarification ($S_{\text{completion}} = 1$) and does not produce a design, S_{design} is set to 1.0, reflecting that abstention is the correct behavior for underspecified prompts.

A.2 RAG Scoring Details

The RAG score is a weighted sum over n evaluated parameters plus a retrieval indicator:

$$S_{\text{RAG}} = \sum_{i=1}^n w_i \cdot a_i^{\text{eff}} + w_{\text{RAG}} \cdot \mathbf{1}(\text{RAG called}) \quad (3)$$

where effective accuracy $a_i^{\text{eff}} = a_i$ if the agent called *search_documents* and 0 otherwise, and raw accuracy $a_i = \mathbf{1}(|v_{\text{actual}} - v_{\text{expected}}| \leq \tau)$ with tolerance $\tau = 0.05$ for volume fraction and force distance, $\tau = 0.5$ for filter radius. The per-prompt scoring weights (Table 7) decrease per-parameter as more parameters are evaluated (1 to 3 across prompts), while w_{RAG} shrinks from 0.40 to 0.10 since the gating mechanism becomes increasingly redundant when multiple parameters must all be correct.

Table 7: RAG scoring weights per prompt. Target parameters are given in Table 4.

ID	w_{vol}	w_{force}	w_{rmin}	w_{rag}
P0	0.60	—	—	0.40
P1	0.40	0.40	—	0.20
P2	0.40	—	0.40	0.20
P3	0.30	0.30	0.30	0.10

A.3 HPC Workflow Score

The primary metric is a weighted composite:

$$S_{\text{HPC}} = w_{\text{step}} \cdot \frac{n_{\text{completed}}}{4} + w_{\text{config}} \cdot c_{\text{config}} + w_{\text{eval}} \cdot \min\left(1, \frac{n_{\text{metrics}}}{6}\right) \quad (4)$$

where $n_{\text{completed}}$ is the number of completed workflow steps (out of 4: generate, submit, monitor, evaluate), $c_{\text{config}} \in \{0, 1\}$ indicates whether the training configuration matches the prompt specifications, and n_{metrics} counts the evaluation metrics successfully extracted. The base weights are $w_{\text{step}} = 0.70$, $w_{\text{config}} = w_{\text{eval}} = 0.15$.

Each secondary weight (w_{config} , w_{eval}) is set to zero when its corresponding step was not called, with the unused weight redistributed to w_{step} (i.e., $w_{\text{step}} = 1.0 - w_{\text{config}} - w_{\text{eval}}$). This avoids double penalization: a missing step is already penalized in the step completion rate. The evaluation metrics counted in n_{metrics} are IOG, COG, FOG, MMD, DPP, and violation rate.

B Generative Model Quality Metrics

For the HPC training benchmark, we report the following EngiOpt metrics [6] to evaluate trained generative models against the dataset baseline. Let $\mathcal{D}$ denote the dataset designs and $\mathcal{D}_g$ the generated designs.

- **MMD** —Distributional similarity between generated and dataset designs (lower is better):

$$\text{MMD}^2(k, \mathcal{D}, \mathcal{D}_g) = \mathbb{E}[k(z, z')] + \mathbb{E}[k(\hat{z}, \hat{z}')] - 2\mathbb{E}[k(z, \hat{z})] \quad (5)$$

where $z, z' \sim \mathcal{D}$, $\hat{z}, \hat{z}' \sim \mathcal{D}_g$, and $k(a, b) = \exp(-\|a - b\|^2 / (2\sigma^2))$ is a Gaussian kernel with $\sigma = 10.0$.

- **DPP** —Diversity of generated designs (higher is better):

$$\text{DPP}(\mathcal{D}_g) = \det(K) \quad (6)$$

where $K \in \mathbb{R}^{M \times M}$ with $K_{ij} = k(\hat{x}_i, \hat{x}_j)$ for $\hat{x}_i, \hat{x}_j \in \mathcal{D}_g$. In practice, K is regularized as $K + 10^{-6}I$ for numerical stability.

- **COG** —Sum of optimality gaps across the optimization path (lower is better):

$$\text{COG}(x_0) = \sum_{t=0}^T [\tilde{f}(x_t, c) - f^*] \quad (7)$$

where $f^* = \max_{x \in \mathcal{X}} \tilde{f}(x, c)$ is the optimal objective value over the feasible design space $\mathcal{X}$, c denotes the conditioning context (e.g., boundary conditions and loads), x_0 is the generated design refined through T optimization steps, and $\tilde{f}$ denotes the simulated objective.

- **RVC** —Fraction of designs violating at least one constraint (lower is better):

$$\text{RVC}(\mathcal{D}_g) = \frac{1}{M} \sum_{k=1}^M \mathbf{1}(\exists i : g_i(\hat{x}_k, a(\hat{x}_k), c) > 0 \text{ or } \exists j : h_j(\hat{x}_k, a(\hat{x}_k), c) \neq 0) \quad (8)$$

where $M = |\mathcal{D}_g|$ is the number of generated designs, $a(\hat{x}_k)$ denotes the attributes of design $\hat{x}_k$, c the conditions, g_i and h_j are inequality and equality constraints, respectively, and $\mathbf{1}(\cdot)$ is the indicator function.

- **IOG** —The $t=0$ term of the COG sum, i.e., $\tilde{f}(x_0, c) - f^*$.
- **FOG** —The $t=T$ term of the COG sum, i.e., $\tilde{f}(x_T, c) - f^*$.

C Benchmark Prompts

Representative prompts for each evaluation dimension. Workflow prompts use Beams2D with volfrac=0.4, forcedist=0.65, rmin=4.0, seed=42; the seven prompt styles target distinct cognitive demands as described in Section 3.4.1. HPC training and RAG evaluation prompts follow.

C.1 Workflow Prompts

FULL

Design a 2D beam structure.

Design requirements:

- Use a material volume fraction of 0.4
- Force distance parameter: 0.65
- Minimum filter radius (rmin): 4.0

Optimize the structure and simulate the result to obtain the compliance value.

NATURAL

Design a 2D beam structure.

Design requirements:

- The design should be moderate material usage
- Apply a force distributed in the upper right region

Optimize the structure and simulate the result to obtain the compliance value.

W-RAND

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration

- Volume Fraction: 0.4
- Force Distance: 0.65
- Filter Radius (rmin): 4.0
- Objective: Minimize compliance

2. Simulation

- After optimization, simulate the design to obtain the compliance value

3. Post-processing & Export

- Thresholding: Apply a 0.58 density threshold to convert the continuous density map into binary geometry
- Mirror: Mirror the design across the y-axis for the final geometry
- XY Scaling: Scale the X and Y dimensions by 2.47
- Extrusion: Extrude the 2D result by 17.9 units in the Z-axis to create a 3D volume
- Export: Save the final geometry as an STL file with these exact parameters

W-DERIVED

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration

- Volume Fraction: 0.4
- Force Distance: 0.65
- Filter Radius (rmin): 4.0
- Objective: Minimize compliance

2. Simulation

- After optimization, simulate the design to obtain the compliance value

3. Post-processing & Export

- The STL export parameters must be derived from the optimization inputs:
- Thresholding: Use the volume fraction value as the density threshold

- Mirror: Mirror the design across the y-axis only if the volume fraction is greater than 0.4
- XY Scaling: Scale the X and Y dimensions by twice the filter radius
- Extrusion: Extrude the 2D result in the Z-axis by the threshold value multiplied by 40
- Export: Save the final geometry as an STL file with these derived parameters

W-DISTRACT

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
2. Simulation
 - After optimization, simulate the design to obtain the compliance value
3. Post-processing & Export
 - Threshold the density field at 0.42 to preview the design topology
 - Apply a 0.58 density threshold to produce the final solid/void geometry
 - Scale the preview display by 1.76x in XY for quick inspection
 - Scale the X and Y dimensions of the part by 2.47 for manufacturing
 - Mirror the design across the y-axis for the final geometry
 - Extrude the 2D result by 17.9 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters

W-COND

Execute a 2D topology optimization, simulate the result, then export the geometry as a 3D-printable STL file with parameters that depend on the simulation outcome.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
2. Simulation
 - After optimization, simulate the design to obtain the compliance value
3. Post-processing & Export (conditional on compliance)
 - If compliance > 254.8:
 - Thresholding: Apply a 0.48 density threshold to convert the continuous density map into binary geometry
 - Mirror: Mirror the design across the y-axis for the final geometry
 - If compliance <= 254.8:
 - Thresholding: Apply a 0.64 density threshold to convert the continuous density map into binary geometry
 - Mirror: Do NOT mirror the design for the final geometry
 - In both cases:
 - XY Scaling: Scale the X and Y dimensions by 0.92
 - Extrusion: Extrude the 2D result by 19.6 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters

W-MULTI

Execute a 2D topology optimization, simulate the result, and export the geometry as TWO separate 3D-printable STL files with different parameters.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
2. Simulation
 - After optimization, simulate the design to obtain the compliance value
3. Post-processing & Export
 - Export A:
 - Thresholding: Apply a 0.48 density threshold to convert the continuous density map into binary geometry
 - Mirror: Mirror the design across the y-axis for the final geometry
 - XY Scaling: Scale the X and Y dimensions by 3.64

- Extrusion: Extrude the 2D result by 19.6 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
- Export B:
- Thresholding: Apply a 0.64 density threshold to convert the continuous density map into binary geometry
 - Mirror: Do NOT mirror the design for the final geometry
 - XY Scaling: Scale the X and Y dimensions by 0.92
 - Extrusion: Extrude the 2D result by 16.4 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters

C.2 HPC Training Prompts

HPC-TRAIN (EXPLICIT)

Train a cGAN CNN 2D generative model for the Beams2D topology optimization problem on the Euler HPC cluster, then evaluate it against the dataset baseline using the standard EngiOpt evaluation script.

Step 1: Generate Training Script

- Use the `generate_training_command` tool with:
 - `algorithm: cgan_cnn_2d`
 - `problem_id: beams2d`
 - `epochs: 100`
 - `seed: 1`

Step 2: Submit to HPC

- Submit the generated SLURM script to the Euler cluster

Step 3: Monitor Training

- Monitor the job until it completes
- Use `check_interval=30` and `max_checks=200` for the monitoring

Step 4: Evaluate Trained Model

- Use the `evaluate_model` tool to evaluate the trained model against the dataset baseline:
 - `problem_id: beams2d`
 - `algorithm: cgan_cnn_2d`
 - `seed: 1`
 - `n_samples: 50`
- This downloads the model from WandB, generates designs, and computes metrics (IOG, COG, FOG, MMD, DPP, violation rate)
- Report the evaluation metrics from the output

Complete all steps in order. Do not ask for clarification.

HPC-TRAIN (NATURAL)

Train a cGAN CNN 2D model for the Beams2D topology optimization problem on the Euler HPC cluster with seed 1 and 100 epochs. Use the available tools to generate the SLURM training script -- do not write or modify any scripts manually. Submit the job and wait for it to finish. Then use the model evaluation tool to evaluate the trained model against the dataset -- it will download the model from WandB automatically. Report the metrics.

Do not ask for clarification.

C.3 RAG Evaluation Prompts

P0

The EngiBench paper documents the default design conditions for the Beams2D problem in its API walkthrough.

Search the paper to find the default volume fraction (`volfrac`) listed for the Beams2D problem. Then generate a 2D beam design using exactly that volume fraction. Use default values for all other parameters (do NOT ask for clarification -- proceed directly with defaults).

P1

In the EngiBench paper's Section 3.1 API walkthrough, a code example runs a Beams2D optimization using non-default design conditions. Search the paper to find both the volume fraction and force distance from that example, then

generate a 2D beam design with those exact values. Use default values for all other parameters and do not ask for clarification.

P2

The SOPTX paper by He et al. (2025) benchmarks its topology optimization framework on a 2D cantilever beam problem.

Search the paper to find both the volume fraction (volfrac) and the filter radius (rmin) used for that 2D cantilever benchmark. Then generate a 2D beam design using exactly those values. Use default values for all other parameters and do not ask for clarification.

P3

Generate a 2D beam design combining parameters from multiple sources:

1. Use the volume fraction and force distance from the EngiBench paper’s API walkthrough example (the non-default values shown in the code snippet).
2. Use the filter radius from the SOPTX paper by He et al. (2025) for their 2D cantilever beam benchmark.

Search the relevant papers to find each value, then generate a 2D beam design using exactly those three parameters. Use default values for all other parameters and do not ask for clarification.

D Supplementary Results

D.1 Diffusion Model Results

Figure 9 shows the offline model quality metrics for agent-trained diffusion models, analogous to the cGAN results in Figure 8. Agent-trained diffusion models achieve comparable values to the EngiBench baselines.

D.2 Photonics2D

Figure 10 provides supplementary visualizations for the Photonics2D W-COND results discussed in Section 4.

E Raw Benchmark Scores

Tables 8 and 9 report all individual sub-scores per model and prompt style, enabling readers to recompute composite scores under alternative weighting schemes.

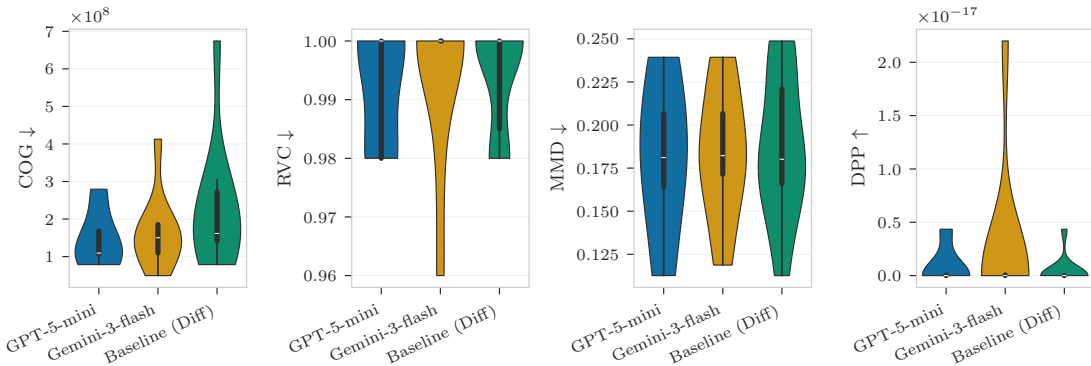
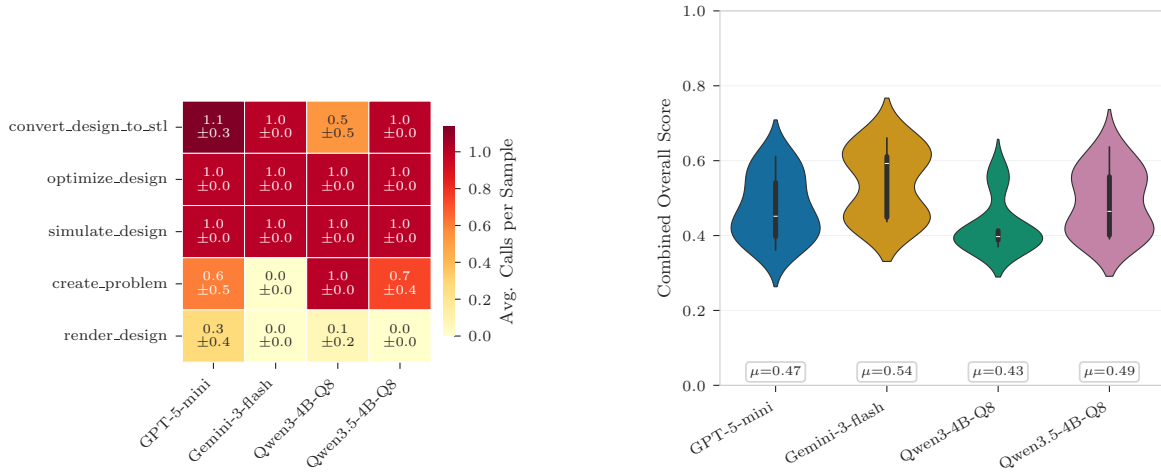


Figure 9: Offline model quality metrics (COG, RVC, MMD, DPP) for agent-trained diffusion models vs. EngiBench baselines. Arrows indicate desired direction. Values averaged across available seeds.



(a) Tool-calling heatmap. All models invoke *convert_design_to_stl* in 100% of runs, confirming that W-COND failures are caused by branch inversion rather than missing tool calls.

(b) Combined overall score distributions. Compared to Beams2D (Figure 4b), all models show broader, lower-scoring distributions, reflecting the increased difficulty of conditional reasoning on an unfamiliar physics domain.

Figure 10: Supplementary Photonics2D W-COND results.

Table 8: Raw per-metric scores (mean $\pm$ std) for all models and prompt styles on Beams2D. Most cells aggregate 15 runs (3 seeds $\times$ 5 samples); a few Qwen3.5-4B configurations have 13–14 runs due to JSON parsing errors. **Bold** = best model per metric within each prompt style. Metric abbreviations are defined in Appendix A. The WT column is identically 0.00 across all models because our threshold-based STL extraction produces non-manifold meshes; this reflects a limitation of the post-processing pipeline rather than agent capability.

Style	Model	IoU $\uparrow$	PA $\uparrow$	Obj $\uparrow$	Constr $\uparrow$	Conn $\uparrow$	WT $\uparrow$	Tool Eff. $\uparrow$	TC $\uparrow$	DQ $\uparrow$	CO $\uparrow$
FULL	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.72 $\pm$ 0.12	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.65 $\pm$ 0.07
	Gemini-3-Flash	0.39 $\pm$ 0.15	0.72 $\pm$ 0.08	0.49 $\pm$ 0.45	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	—	0.98 $\pm$ 0.08	0.93 $\pm$ 0.25	0.54 $\pm$ 0.09	0.69 $\pm$ 0.10
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.16 $\pm$ 0.33	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	—	0.67 $\pm$ 0.00	0.00 $\pm$ 0.00	0.49 $\pm$ 0.10	0.45 $\pm$ 0.06
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.37 $\pm$ 0.46	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	—	0.72 $\pm$ 0.05	0.73 $\pm$ 0.44	0.52 $\pm$ 0.09	0.59 $\pm$ 0.11
NATURAL	GPT-5-mini	0.39 $\pm$ 0.14	0.73 $\pm$ 0.05	0.04 $\pm$ 0.06	0.09 $\pm$ 0.12	0.75 $\pm$ 0.43	0.00 $\pm$ 0.00	0.75 $\pm$ 0.41	0.87 $\pm$ 0.34	0.37 $\pm$ 0.11	0.82 $\pm$ 0.30
	Gemini-3-Flash	0.34 $\pm$ 0.07	0.69 $\pm$ 0.05	0.10 $\pm$ 0.14	0.13 $\pm$ 0.11	0.67 $\pm$ 0.47	—	0.81 $\pm$ 0.33	1.00 $\pm$ 0.00	0.35 $\pm$ 0.07	0.88 $\pm$ 0.23
	Qwen3-4B	0.46 $\pm$ 0.18	0.72 $\pm$ 0.12	0.24 $\pm$ 0.32	0.05 $\pm$ 0.10	1.00 $\pm$ 0.00	—	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.44 $\pm$ 0.07	0.29 $\pm$ 0.05
	Qwen3.5-4B	0.47 $\pm$ 0.13	0.74 $\pm$ 0.09	0.13 $\pm$ 0.27	0.06 $\pm$ 0.11	0.91 $\pm$ 0.29	—	0.28 $\pm$ 0.44	0.33 $\pm$ 0.47	0.42 $\pm$ 0.07	0.48 $\pm$ 0.32
W-RAND	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.77 $\pm$ 0.17	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.66 $\pm$ 0.07
	Gemini-3-Flash	0.40 $\pm$ 0.11	0.73 $\pm$ 0.06	0.35 $\pm$ 0.44	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.80 $\pm$ 0.18	1.00 $\pm$ 0.00	0.52 $\pm$ 0.09	0.65 $\pm$ 0.09
	Qwen3-4B	0.39 $\pm$ 0.15	0.72 $\pm$ 0.08	0.48 $\pm$ 0.45	1.00 $\pm$ 0.00	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.55 $\pm$ 0.11	0.64 $\pm$ 0.07
	Qwen3.5-4B	0.40 $\pm$ 0.16	0.73 $\pm$ 0.08	0.38 $\pm$ 0.44	1.00 $\pm$ 0.00	0.71 $\pm$ 0.45	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.08	0.69 $\pm$ 0.06
W-DERIVED	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.76 $\pm$ 0.14	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.65 $\pm$ 0.07
	Gemini-3-Flash	0.41 $\pm$ 0.13	0.74 $\pm$ 0.07	0.36 $\pm$ 0.43	0.93 $\pm$ 0.25	0.53 $\pm$ 0.50	0.00 $\pm$ 0.00	0.81 $\pm$ 0.14	1.00 $\pm$ 0.00	0.50 $\pm$ 0.11	0.64 $\pm$ 0.07
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.28 $\pm$ 0.40	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.64 $\pm$ 0.07	0.47 $\pm$ 0.50	0.51 $\pm$ 0.08	0.53 $\pm$ 0.08
	Qwen3.5-4B	0.41 $\pm$ 0.16	0.73 $\pm$ 0.09	0.43 $\pm$ 0.46	1.00 $\pm$ 0.00	0.62 $\pm$ 0.49	0.00 $\pm$ 0.00	0.95 $\pm$ 0.18	0.85 $\pm$ 0.36	0.52 $\pm$ 0.09	0.66 $\pm$ 0.10
W-DISTRACT	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.63 $\pm$ 0.08	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.63 $\pm$ 0.06
	Gemini-3-Flash	0.41 $\pm$ 0.14	0.74 $\pm$ 0.07	0.40 $\pm$ 0.43	1.00 $\pm$ 0.00	0.53 $\pm$ 0.50	0.00 $\pm$ 0.00	0.72 $\pm$ 0.06	1.00 $\pm$ 0.00	0.51 $\pm$ 0.11	0.63 $\pm$ 0.08
	Qwen3-4B	0.39 $\pm$ 0.15	0.73 $\pm$ 0.08	0.41 $\pm$ 0.44	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.60 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.10	0.61 $\pm$ 0.06
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.42 $\pm$ 0.45	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.88 $\pm$ 0.12	0.93 $\pm$ 0.25	0.53 $\pm$ 0.10	0.66 $\pm$ 0.09
W-COND	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.84 $\pm$ 0.16	0.93 $\pm$ 0.25	0.54 $\pm$ 0.10	0.66 $\pm$ 0.10
	Gemini-3-Flash	0.40 $\pm$ 0.16	0.73 $\pm$ 0.08	0.40 $\pm$ 0.43	0.93 $\pm$ 0.25	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.86 $\pm$ 0.17	0.87 $\pm$ 0.34	0.54 $\pm$ 0.10	0.65 $\pm$ 0.10
	Qwen3-4B	0.39 $\pm$ 0.15	0.72 $\pm$ 0.09	0.48 $\pm$ 0.44	1.00 $\pm$ 0.00	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.76 $\pm$ 0.15	0.40 $\pm$ 0.49	0.55 $\pm$ 0.10	0.57 $\pm$ 0.12
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.33 $\pm$ 0.41	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.60 $\pm$ 0.49	0.51 $\pm$ 0.10	0.62 $\pm$ 0.13
W-MULTI	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.87 $\pm$ 0.13	0.93 $\pm$ 0.25	0.54 $\pm$ 0.10	0.66 $\pm$ 0.09
	Gemini-3-Flash	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.99 $\pm$ 0.05	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.70 $\pm$ 0.07
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.34 $\pm$ 0.41	1.00 $\pm$ 0.00	0.80 $\pm$ 0.40	0.00 $\pm$ 0.00	0.78 $\pm$ 0.05	1.00 $\pm$ 0.00	0.53 $\pm$ 0.11	0.65 $\pm$ 0.08
	Qwen3.5-4B	0.42 $\pm$ 0.14	0.74 $\pm$ 0.07	0.42 $\pm$ 0.42	1.00 $\pm$ 0.00	0.64 $\pm$ 0.48	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.10	0.69 $\pm$ 0.06

Table 9: Raw per-metric scores (mean $\pm$ std) for all models and prompt styles on Photonics2D. Most cells aggregate 15 runs (3 seeds $\times$ 5 samples); a few Qwen3.5-4B configurations have 13–14 runs due to JSON parsing errors. **Bold** = best model per metric within each prompt style. Metric abbreviations are defined in Appendix A. The WT column is identically 0.00 across all models because our threshold-based STL extraction produces non-manifold meshes; this reflects a limitation of the post-processing pipeline rather than agent capability.

Style	Model	IoU $\uparrow$	PA $\uparrow$	Obj $\uparrow$	Constr $\uparrow$	Conn $\uparrow$	WT $\uparrow$	Tool Eff. $\uparrow$	TC $\uparrow$	DQ $\uparrow$	CO $\uparrow$
W-RAND	GPT-5-mini	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.53 $\pm$ 0.01
	Gemini-3-Flash	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.01 $\pm$ 0.03	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.97 $\pm$ 0.08	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.60 $\pm$ 0.02
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.14	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.75 $\pm$ 0.00	1.00 $\pm$ 0.00	0.39 $\pm$ 0.03	0.56 $\pm$ 0.02
	Qwen3.5-4B	0.31 $\pm$ 0.06	0.89 $\pm$ 0.01	0.03 $\pm$ 0.10	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.73 $\pm$ 0.05	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.55 $\pm$ 0.01
W-DISTRACT	GPT-5-mini	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.02 $\pm$ 0.06	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.59 $\pm$ 0.07	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.52 $\pm$ 0.02
	Gemini-3-Flash	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.05 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.72 $\pm$ 0.06	1.00 $\pm$ 0.00	0.40 $\pm$ 0.04	0.55 $\pm$ 0.03
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.14	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.58 $\pm$ 0.04	1.00 $\pm$ 0.00	0.39 $\pm$ 0.03	0.52 $\pm$ 0.02
	Qwen3.5-4B	0.32 $\pm$ 0.06	0.89 $\pm$ 0.01	0.05 $\pm$ 0.19	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.82 $\pm$ 0.16	1.00 $\pm$ 0.00	0.40 $\pm$ 0.04	0.57 $\pm$ 0.05
W-COND	GPT-5-mini	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.06 $\pm$ 0.16	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.78 $\pm$ 0.16	0.40 $\pm$ 0.49	0.40 $\pm$ 0.03	0.47 $\pm$ 0.08
	Gemini-3-Flash	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.08 $\pm$ 0.17	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.50	0.40 $\pm$ 0.03	0.54 $\pm$ 0.09
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.70 $\pm$ 0.05	0.20 $\pm$ 0.40	0.39 $\pm$ 0.03	0.43 $\pm$ 0.07
	Qwen3.5-4B	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.82 $\pm$ 0.11	0.47 $\pm$ 0.50	0.39 $\pm$ 0.04	0.49 $\pm$ 0.08